

SKILLSIGNALZA

Software Engineering Career Map Pack

A South African entry-level route guide

Choose a lane. Build depth in one stack. Produce visible proof. Apply with a clear target.

VERSION 1.0 | AUGUST 2026
Market evidence cutoff: 11 August 2026

START HERE

What this pack will help you finish

This pack turns a broad interest in software development into a working career route. It gives you a lane, a stack boundary, project standards, evidence checks, application filters and a 30/60/90-day plan.

By the final page, you will have completed these decisions:

- The Software Engineering work you want to target first.
- The job titles that belong in your search.
- The primary language and stack for your current build cycle.
- The project that will carry your strongest application evidence.
- The completion gates that determine when you advance.
- The application routine you will use to find recurring gaps.

WORKING PROMISE: Use the worksheets and completion gates. You will leave with a focused route and a defensible next action.

Product boundary

Included in this edition:

- South African entry-level market direction.
- Software Engineering lane selection.
- Skill priorities and stack recommendations.
- Project blueprints with acceptance checks.
- Evidence packaging and application planning.

Outside this edition:

- A personalised readiness score.
- Individual CV review or rewriting.
- Vacancy matching and live job feeds.
- Salary forecasting or recruitment services.
- Job, interview or hiring guarantees.

NAVIGATION

Contents

Start here	2	Back-end integration	21
How to use the pack	4	AI-adjacent extension	23
Starting point	5	Lane decision worksheet	24
Foundation Builder	7	Milestone routes	25
Evidence Builder	8	Project blueprints	29
Market evidence	9	Package and apply	33
Shared foundations	12	30/60/90-day route	37
Career lanes	16	Personal execution plan	38
Application and full-stack	17	Research and version policy	39
Front-end interface	19		

Recommended reading order

Complete Starting Point before choosing a lane. Read the full Market Evidence section. Compare every lane once, then commit to one lane for the current evidence cycle. Use the project and application sections with that choice in view.

KEEP ONE WORKING DIRECTION: You can change lanes after a completed project and an application review. Switching during the build usually leaves every stack shallow.

HOW TO USE THE PACK

Keep an editable copy or notebook open while reading. Every section should change a decision, a build task or an application action.

- 1 Record your current evidence before studying new tools.
- 2 Select a starting route based on what you can already control.
- 3 Choose a lane and define the stack boundary for this cycle.
- 4 Build or repair one project against its completion gate.
- 5 Package the same evidence consistently across your CV and links.
- 6 Start targeted applications and review the patterns every week.

The evidence ladder

Use four levels when judging a skill:

Level	Evidence standard	Direct check
Named	The skill appears in your CV or learning record.	Point to the exact claim.
Explained	You can describe its purpose and where it fits.	Explain it in plain language.
Applied	You used it to complete a bounded requirement.	Show the task and your implementation.
Demonstrated	Another person can inspect the output and your contribution.	Open the repository, demo or work record.

A long skills list gives you named evidence. Strong applications depend on applied and demonstrated evidence for the core stack.

Course rule

Use one structured learning resource for each primary skill. Keep official documentation nearby. Move into exercises and project work while the material is still fresh. A certificate can support the application; working evidence carries the technical claim.

STARTING POINT

Record what already exists

Write from evidence rather than memory. Include coursework, internship tasks, volunteer systems, freelance work, workplace improvements and personal tools.

Prompt	Your answer
Current education, training or technical experience	
Languages used to complete real tasks	
Projects that still run or can be recovered	
Links another person can access	
Weekly time available for focused work	
Closest target job title	
Current blockage	Knowledge / practice / proof / application

Run a four-part foundation check

- Programming: write a small feature from a requirement and explain the control flow.
- Data: create related records, query them and explain the relationship.
- Debugging: reproduce a defect, isolate the cause and verify the repair.
- Git: create a branch, make clear commits, review the diff and merge the work.

Score each item as Named, Explained, Applied or Demonstrated. Your lowest core items determine the correct starting route.

PLACEMENT RULE: Choose Foundation Builder when core work breaks down before the Applied level. Choose Evidence Builder when the work is usable and the visible proof remains weak.

STARTING POINT

Choose your entry route

Current position	Begin here	Immediate priority
Core programming still depends on full tutorials.	Foundation Builder	Gain control of one language and small requirements.
You can build features, while your work is private, scattered or unclear.	Evidence Builder	Recover, strengthen and publish the best evidence.
Strong work exists and the application story is inconsistent.	Application Packaging	Align the CV, repository and target role.
Evidence and applications are active.	Application Loop	Track responses and close recurring gaps.

A direct placement test

Attempt one feature in your intended stack without a step-by-step tutorial. Use documentation and search as working tools. Plan the data, implement the main path, trigger one failure and commit the repair.

Begin with Evidence Builder when you can structure the work and recover from ordinary errors. Begin with Foundation Builder when the feature remains dependent on copied structure.

My route for this cycle	Evidence supporting the choice

START HONESTLY: A lower starting point protects you from producing a project you cannot explain.

FOUNDATION BUILDER

Establish independent control

Your target is a small integrated system built from written requirements. Mastery can wait. You need enough control to plan a feature, connect the parts, diagnose ordinary errors and explain your choices.

Work in this order:

- 1 Select the lane and primary language.
- 2 Practise the language through short tasks with visible outputs.
- 3 Add the data, API or browser layer required by the lane.
- 4 Use Git throughout a small integrated build.
- 5 Complete the main project workflow and expected failure paths.
- 6 Document the decisions and package the evidence.

Progress signals:

- You can predict the result of a short code path.
- You can split a requirement into implementable tasks.
- You can read an error and test a likely cause.
- You can adapt documentation examples to your own data.
- You can explain how the interface, logic and storage connect.
- You can identify code you would improve in a second version.

COMPLETION GATE: Start the main lane project when you can build one small feature from a written requirement, commit the work and explain the data flow.

EVIDENCE BUILDER

Repair the shortest path to credibility

Recover useful work before starting another course. Select the project with the strongest lane fit and a realistic repair scope.

Audit every candidate project:

- The link opens and the setup instructions work.
- The repository belongs to you or states your contribution accurately.
- The main workflow reaches a clear result.
- The code contains enough lane-specific depth.
- The history shows staged development.
- Expected failures receive deliberate handling.
- Tests, screenshots, sample requests or a demo make inspection easier.
- The README explains the problem, scope, stack and decisions.

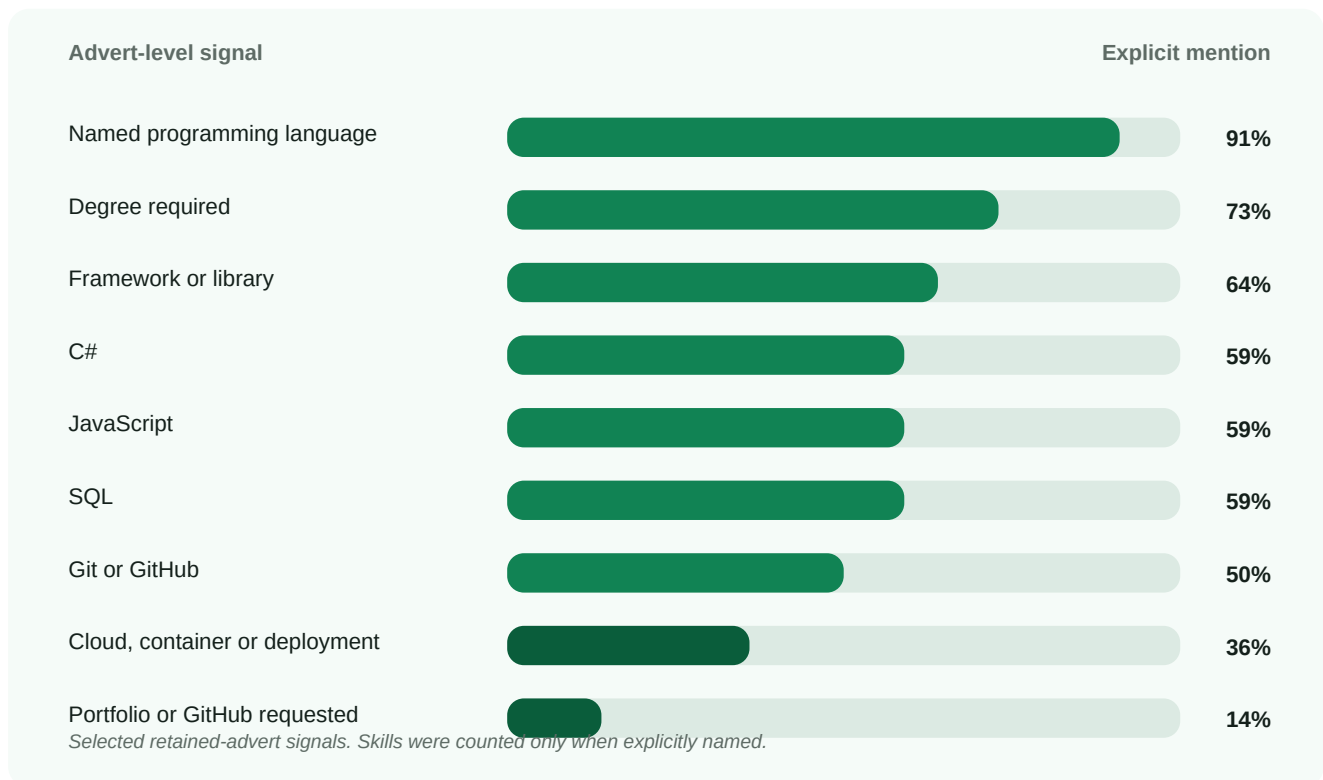
Repair in this order:

- 1 Restore access and repeatable setup.
- 2 Clarify ownership and reused material.
- 3 Add the missing lane-specific proof.
- 4 Improve validation, errors, logs or tests.
- 5 Rewrite the technical explanation.
- 6 Add visual polish or deployment after the core flow is stable.

COMPLETION GATE: Move to application packaging when another person can access the work, understand the problem and identify what you personally built.

MARKET EVIDENCE

What appeared in the retained Software Engineering adverts



The chart shows explicit advert wording. It supports priority decisions rather than a universal technology ranking.

Direct readings

- A named programming language appeared in 91% of the retained adverts. Your application needs one language supported by context or visible work.
- Degree requirements appeared in 73%. Qualifications affect access to advertised roles.
- A framework or library appeared in 64%. Framework evidence strengthens a stack after the parent language is usable.
- C#, JavaScript and SQL each appeared in 59%. Their frequency supports practical application and web routes.
- Git or GitHub appeared in 50%. Use Git as a working habit and show the workflow.
- Cloud, container or deployment tools appeared in 36%. Add them after the application works.
- Portfolio or GitHub proof was explicitly requested in 14%. A project remains useful for proving claims when professional experience is limited.

RESEARCH USE: Frequencies describe the retained evidence. Your lane and target adverts determine the final stack.

MARKET EVIDENCE

Turn frequency into a build order

Priority	Treatment	Software Engineering examples
Foundation	Build and evidence before broad applications.	One language, programming concepts, debugging, data fundamentals.
Work-ready layer	Connect to the chosen lane and project.	Git, relevant framework, APIs, browser behaviour, basic testing.
Differentiator	Add after the core workflow works.	Deployment, cloud exposure, containers, stronger test coverage.
Target-specific	Learn when target adverts repeatedly request it.	A second framework, vendor platform, specialist runtime or legacy stack.

The default route for an undecided beginner

Use C# + .NET + SQL + JavaScript + HTML/CSS + Git. Build one database-backed web application.

This route follows the strongest Microsoft-oriented pattern in the retained evidence and supports Junior Software Developer, Application Developer and Junior Full Stack searches.

Choose a different route when your target lane is clear:

- Front-end: JavaScript, HTML and CSS first. Add TypeScript and React.
- Back-end: C# with .NET for the default route. Java is a viable alternative; select its framework from repeated target adverts.
- AI-adjacent extension: Python, APIs, structured data, Git and evaluation inside a working service.
- Target-specific server-side web: Use PHP with the database and operating environment named in repeated target adverts.

Build during this cycle	Park for later

MARKET EVIDENCE

Qualifications, experience and location

Qualification wording is an access filter. Technical evidence serves a different job: it shows what you can do.

Observed direction:

- Degree requirements were the dominant qualification pattern.
- Six retained adverts stated no qualification requirement.
- Most experience wording sat between zero and two years.
- Hybrid roles formed 67% of known arrangements.
- Gauteng accounted for 73% of retained roles.
- Remote roles formed 10% of known arrangements.

Application rules

- Meet a required qualification or use an advert that explicitly accepts equivalent experience.
- Prioritise graduate, internship and junior routes when your professional history is short.
- Apply to zero-to-two-year roles once the lane, core stack and project evidence align.
- Read two-plus-year adverts carefully. Apply when your real experience covers the responsibilities.
- Put Gauteng and Cape Town into the core search. Add selected KwaZulu-Natal routes where the lane matches.
- Treat remote-only searching as a narrow strategy. Keep hybrid and on-site filters active when practical.
- When the title and requirement paragraph conflict, judge the responsibilities and seniority evidence inside the advert.

QUALIFICATION DECISION: Missing a degree narrows access. Build evidence while concentrating applications on equivalent-experience, no-qualification, internship and skills-based routes.

SHARED FOUNDATIONS

Build one primary language deeply enough to work

Choose the language from the lane. Keep it through the full project cycle.

Lane	Primary choice	Evidence it should carry
Application and full-stack	C# with .NET	Application logic, API behaviour, validation and database work.
Front-end interface	JavaScript, then TypeScript	Browser behaviour, components, state and API consumption.
Back-end integration	C# with .NET; Java with the framework named in target adverts	Services, data integrity, contracts and integration logic.
AI-adjacent extension	Python	Structured data, evaluation and service integration.

Depth standard:

- Use variables, conditions, loops, functions and collections deliberately.
- Model a small domain with suitable classes or modules.
- Validate input and handle expected failures.
- Read stack traces or error output and isolate likely causes.
- Work with HTTP, files or a database where the lane requires it.
- Explain one design decision and the trade-off you accepted.
- Refactor a weak implementation without changing the intended result.
- Write a small test around a core rule.

STOP SWITCHING: Keep the primary language until you can build and explain a small system. Add a second language only for a repeated target-role need or a clear project requirement.

SHARED FOUNDATIONS

Data, APIs and system flow

Entry-level applications become credible when data moves through a complete workflow.

Database standard:

- Create tables or collections that match the problem domain.
- Define identifiers, relationships and required fields deliberately.
- Write purposeful create, read, update and delete operations.
- Use filtering, joins or aggregation where the problem needs them.
- Reject invalid data close to the rule that protects it.
- Seed sample data so another person can inspect the system quickly.

API and integration standard:

- Define clear request and response shapes.
- Use suitable status codes and consistent error messages.
- Separate input validation from business rules.
- Protect configuration values and secrets.
- Handle timeouts, invalid responses and unavailable dependencies.
- Document sample requests or expose an inspectable contract.

System explanation check:

- 1 Where does input enter?
- 2 Which code validates it?
- 3 Where do the business rules run?
- 4 How is data stored or retrieved?
- 5 What reaches the user when the operation fails?
- 6 Which log, test or output verifies the result?

COMPLETION GATE: You can trace one main workflow from input to stored data and back to a visible result.

SHARED FOUNDATIONS

Debugging, testing and code quality

Debugging evidence shows control over your own implementation. Testing strengthens that evidence when it targets important behaviour.

Use this debugging record during the project:

Field	Record
Symptom and exact reproduction step	
Evidence gathered	
Likely cause tested	
Repair made	
Verification after repair	
Regression check added	

Minimum testing standard:

- Test one central business rule.
- Test an invalid input or blocked state change.
- Test a failure from an external or data dependency.
- Test the main interface or endpoint path where practical.
- Keep test names readable enough to describe the behaviour.
- Document manual verification when automation would exceed the project scope.

Code review checklist:

- Names match the domain.
- Functions and components have focused responsibilities.
- Repeated logic has a clear reason or a shared implementation.
- Errors reach the correct layer.
- Sensitive values stay outside source control.
- Comments explain decisions rather than restating the code.

SHARED FOUNDATIONS

Git, collaboration and deployment

Git appeared in half of the retained adverts. Use it throughout the build so the repository shows working practice.

Repository workflow:

- 1 Create a focused issue or task.
- 2 Work on a short-lived branch.
- 3 Commit coherent changes with specific messages.
- 4 Review the diff before merging.
- 5 Record the decision or defect in the issue or README.
- 6 Tag a stable version when the minimum workflow passes.

Good commit messages describe the change:

- Add server validation for invalid status transitions
- Handle failed application-list requests in the interface
- Create seeded PostgreSQL schema for service requests
- Add classifier fallback below the confidence threshold

Deployment comes after local reliability. A deployed broken workflow weakens the evidence.

Deployment readiness:

- Configuration values load from the environment.
- Setup works from a clean copy.
- Seed data or a demo account is available.
- Health checks and basic logs expose failure.
- The README names deployment limits and costs.
- Private data stays outside the public repository.

PRIORITY: Finish the workflow locally, test the expected failures, then publish a stable version.

CAREER LANES

Choose the work before the tool list

Lane	Work focus	Strongest project evidence	Main debugging surface
Application and full-stack	Complete operational workflows	Integrated database-backed application	Across interface, server and data
Front-end interface	Interaction, state and accessibility	Polished responsive interface using realistic data	Browser, network and state
Back-end integration	Rules, APIs, data integrity and services	Documented API with tests and controlled failures	Requests, data, logs and dependencies
AI-adjacent extension	Evaluated intelligent feature added after the shared foundation	Working service plus evaluation and fallback	Input, output, evaluation and service behaviour

Choose the lane whose work can hold your attention through debugging and revision.

Decision prompts:

- Which layer do I inspect first when a system fails?
- Which output would I enjoy improving for several weeks?
- Which lane fits the strongest work I can already recover?
- Which titles appear repeatedly in the locations I can accept?
- Which stack can I keep stable through one project cycle?
- Which project can I explain without inflating my contribution?

COMMITMENT: Keep the selected lane through one project, one packaging pass and one targeted application batch.

LANE 1: APPLICATION AND FULL-STACK DEVELOPMENT

Build complete business workflows

This lane suits candidates who enjoy following a real process across screens, rules and stored data.

Typical work:

- Turn operational requirements into application behaviour.
- Build forms, validation and role-aware workflows.
- Create APIs or server actions that enforce business rules.
- Model data and maintain reliable state changes.
- Investigate defects across several layers.
- Improve existing systems after user feedback.

Observed title family:

- Junior Software Developer
- Application Developer
- Junior Full Stack Developer
- Junior Software Engineer
- Software Engineering Intern
- Graduate Developer

Recommended core stack:

C# + .NET + SQL + JavaScript + HTML/CSS + Git

Add React or Angular when the interface needs component depth. Add Azure or Docker after the core application works.

Choose this lane when:

- A complete user workflow interests you more than one isolated layer.
- You want to connect interface behaviour with data rules.
- You can tolerate debugging across several parts of the system.
- Business applications and internal tools feel meaningful to you.

LANE 1: APPLICATION AND FULL-STACK DEVELOPMENT

Skill and evidence plan

Layer	Build standard	Visible evidence
Language and framework	C# application structure, dependency use and clear domain logic	Code that separates interface, rules and persistence
Data	Relational model, constraints and purposeful queries	Schema, migrations and seeded records
Server behaviour	Validated operations and consistent error handling	API contract or documented server actions
Interface	Usable forms, filtering and full request states	Screenshots or demo of normal and failed paths
Verification	Tests around central rules and invalid changes	Test output and a short verification note
Delivery	Repeatable setup and stable release	README, configuration guide and deployed demo where practical

Completion gate:

- A user can complete the main workflow from a clean start.
- Invalid input receives a specific response.
- The data model preserves required relationships.
- Expected failures stay visible and recoverable.
- Another developer can follow the setup instructions.
- You can explain the boundaries between each layer.

Search terms:

Junior Software Developer; Junior .NET Developer; Application Developer; Junior Full Stack Developer; Graduate Software Engineer; Software Development Intern.

LANE 2: FRONT-END AND WEB INTERFACE DEVELOPMENT

Build interfaces that communicate system state clearly

This lane suits candidates who care about interaction, layout, accessibility and the behaviour between user action and application response.

Typical work:

- Build responsive screens from written requirements or designs.
- Manage component state and data loaded from APIs.
- Handle loading, empty, success, error and permission states.
- Create reusable interface patterns.
- Investigate browser, network and rendering problems.
- Protect accessibility across keyboard and small-screen use.

Recommended sequence:

JavaScript + HTML + CSS -> TypeScript -> React -> testing and deployment

Learn browser fundamentals before depending on a framework. Add React after you can handle events, forms, DOM behaviour, asynchronous requests and responsive layout directly.

Choose this lane when:

- Interface behaviour holds your attention.
- You notice unclear states and awkward interaction.
- You enjoy translating requirements into visible behaviour.
- You want users to feel the result of your technical decisions immediately.

Observed title family:

Junior Front-End Developer; Junior Web Developer; Junior React Developer; UI Developer; Web Development Intern; Front-End Software Engineer.

LANE 2: FRONT-END AND WEB INTERFACE DEVELOPMENT

Skill and evidence plan

Layer	Build standard	Visible evidence
Browser foundation	Semantic HTML, responsive CSS and JavaScript events	Working screens without framework dependence
Component code	Typed inputs, focused responsibilities and reusable patterns	Readable TypeScript and component boundaries
Data flow	API consumption, cancellation or stale-state control where needed	Main request flow with error handling
Accessibility	Labels, focus states, keyboard paths and suitable contrast	Accessibility notes and manual or automated checks
Verification	Interaction tests for important user behaviour	Test output and explained coverage choices
Delivery	Stable build, environment configuration and performance awareness	Deployed interface or recorded demonstration

Completion gate:

- The main workflow works on narrow and wide screens.
- Every asynchronous state produces a clear response.
- Keyboard users can complete the central task.
- Components have readable responsibilities.
- Failed requests produce recovery options.
- The README explains the interface decisions and known limits.

Priority order:

- 1 Semantic HTML and responsive CSS.
- 2 JavaScript behaviour and asynchronous requests.
- 3 TypeScript and component architecture.
- 4 Accessibility and interaction testing.
- 5 Deployment, performance and reusable design patterns.

LANE 3: BACK-END AND SYSTEMS INTEGRATION

Build services that protect rules and data

This lane suits candidates who enjoy tracing requests, preserving data integrity and controlling failure across dependencies.

Typical work:

- Design APIs and server-side application behaviour.
- Model persistent data and enforce valid changes.
- Integrate external services with defensive handling.
- Investigate logs, defects and performance problems.
- Implement authentication or authorisation where required.
- Support deployment and operational visibility.

Choose one stack route:

Route	Core combination	Evidence position
Microsoft service	C# + .NET + SQL + Git	C#: 13/22 adverts; SQL: 13/22; .NET: 10/22.
Java service	Java + SQL + Git; add the target framework	Java: 10/22 adverts. Spring: 1/22.

Use the Microsoft service route when your target list lacks a stronger repeated pattern.

PHP appeared in 3 of 22 retained adverts. MySQL and Linux each appeared in 3. The full PHP + MySQL + Linux + Git combination appeared in one retained advert. Build that route for a target list that repeats the combination.

Observed title family:

Junior Back-End Developer; Junior Java Developer; Junior .NET Developer; Software Developer; API Developer; Integration Developer; Graduate Software Engineer.

LANE 3: BACK-END AND SYSTEMS INTEGRATION

Skill and evidence plan

Minimum project proof:

- A documented API with clear request and response contracts.
- Persistent data with deliberate relationships and constraints.
- Input validation and server-enforced business rules.
- Consistent error responses and useful logs.
- Tests for central rules and invalid paths.
- Repeatable setup with seeded data and sample requests.
- Authentication or permissions where the problem requires them.
- Defensive handling around one simulated external dependency.

Completion gate:

Another developer can start the service, run sample requests, inspect the data model, trigger a controlled failure and find the matching test or log.

Depth sequence:

- 1 Language and framework control.
- 2 HTTP, routing and request validation.
- 3 Relational data and migrations.
- 4 Business rules and error contracts.
- 5 Automated tests and structured logs.
- 6 Authentication, deployment and integration resilience.

Avoid spending the first weeks on microservices, queues or elaborate infrastructure. A clear single service with strong boundaries provides better entry-level evidence.

AI-ADJACENT EXTENSION

Add an evaluated intelligent feature after the software foundation

Four of the 22 retained adverts explicitly referenced AI work in the job title or captured requirements. This evidence base is smaller than the core software lanes. Complete the shared Software Engineering foundation before adding a focused feature.

Entry gate:

- Python used beyond copied notebooks.
- API fundamentals and structured data handling.
- Git workflow and a readable repository.
- Testing, logging and controlled failure behaviour.
- Basic deployment or cloud awareness.
- A defined evaluation method for the feature output.

Strong evidence shows:

- The feature has one clearly defined job.
- Inputs and outputs use an inspectable format.
- Evaluation uses held-out examples or a suitable benchmark.
- Low-confidence output follows a review or fallback path.
- Logs avoid sensitive source text by default.
- The software around the feature remains reliable.
- Error analysis names common failure patterns.
- Your own decisions are separated from third-party model behaviour.

Recommended route:

Python + API service + structured data + Git + tests + evaluation + basic deployment

ROUTE STATUS: Use this extension for AI-labelled target roles whose general software requirements match your foundation.

Search terms:

AI Software Engineer Junior/Graduate; Software Development Intern AI/Data; Junior Software Developer AI Pathway; Python Developer; Cloud Product Intern.

LANE DECISION WORKSHEET

Commit for one evidence cycle

Decision	Your answer
Selected lane	
Work in this lane that attracts me	
Target job titles	
Primary language	
Framework or runtime	
Database or data store	
Project I will build or repair	
Weekly hours I can protect	
Evidence I expect to publish	
Date for the first project review	

Stack boundary

Build now:

Park for later:

Evidence reason

Write four sentences covering the target role, repeated advert signals, the project workflow and the current evidence gap.

REVIEW POINT: Revisit the lane after the project completion gate and the first targeted application batch.

MILESTONE ROUTES

Foundation Builder: establish control

Milestone 1: route clarity

Requirement:

- Describe the lane's work and the purpose of every tool in the chosen stack.
- Collect target titles and repeated requirements from current adverts.
- Record the stack boundary and weekly schedule.
- Select the project blueprint.

Suggested window: 2 to 5 days.

Milestone 2: language control

Requirement:

- Explain core syntax, program flow, functions and common data structures.
- Complete small written requirements without a full solution guide.
- Debug ordinary mistakes using error output and documentation.
- Publish a short repository of purposeful exercises or a small utility.

Suggested window: 1 to 3 weeks, adjusted for prior experience and weekly hours.

Advance when

You can read a small requirement, plan the program, implement the main path, recover from expected errors and explain the result.

Milestone	Target date	Evidence link	Complete
Route clarity			[]
Language control			[]

MILESTONE ROUTES

Foundation Builder: connect and publish

Milestone	Completion gate	Planning window
Connected stack	Two or more layers exchange valid data and expected failures receive handling.	2 to 4 weeks
Visible project	The main user workflow works and the repository explains setup and decisions.	2 to 4 weeks
Evidence packaged	CV, repository and project summary describe the same contribution accurately.	3 to 7 days
Application loop	Targeted search, tracker and weekly gap review are active.	Ongoing

Review questions:

- Can I explain how the parts connect?
- Can I complete the main workflow from a clean start?
- Can I show evidence for the skills named on my CV?
- Can I describe one technical decision and one repaired failure?
- Can another person access the work from the supplied link?
- Can I state the current limits without blaming unfinished scope?

When progress stalls, reduce feature scope and keep the completion standard. Preserve one stretch feature outside the minimum version. Add it after the central workflow passes.

TIME RULE: Planning windows help you schedule. Completion gates control progression.

MILESTONE ROUTES

Evidence Builder: select the repair candidate

Project	Lane fit	Runs	Accessible	Ownership clear	Repair value
	High / Medium / Low	Yes / No	Yes / No	Yes / No	High / Medium / Low
	High / Medium / Low	Yes / No	Yes / No	Yes / No	High / Medium / Low
	High / Medium / Low	Yes / No	Yes / No	Yes / No	High / Medium / Low
	High / Medium / Low	Yes / No	Yes / No	Yes / No	High / Medium / Low

Select the project with:

- Strong relevance to the chosen lane.
- A working central workflow or recoverable codebase.
- Enough ownership to explain the important decisions.
- A repair scope that fits the current cycle.
- Evidence that can become public without exposing sensitive data.
- A clear path to one stronger technical proof point.

Decision	Your answer
Project I will repair or replace	
Missing proof weakening it	
Smallest change that closes the gap	
Evidence I will archive	

DISCARD DELIBERATELY: Archive broken links, copied tutorial clones and projects unrelated to the selected lane.

MILESTONE ROUTES

Evidence Builder: repair in the right order

Order	Repair	Done when
1	Access and setup	The link works and a clean environment follows the instructions.
2	Ownership and scope	Your role, contribution and reused material are described accurately.
3	Core lane proof	The project demonstrates the lane's main workflow and technical layer.
4	Failure quality	Validation, errors, logs or tests cover expected failures.
5	Explanation	README and CV entry name the problem, stack, decisions and result.
6	Inspection polish	Screenshots, demo or deployment improve review without hiding defects.

Selective gap closure:

Learn a new tool only when the repair needs it or repeated target adverts make it material. Define one narrow output, such as a tested endpoint or a TypeScript conversion of an important feature. Stop when the evidence gap closes.

Completion check:

- The strongest project is accessible and reproducible.
- The repository tells the same story as the CV.
- Working code or output carries the lane's core claim.
- Known limits are stated clearly.
- You can explain the work in four minutes.
- Follow-up questions can reach implementation detail.

PROJECT BLUEPRINT 1

Operations workflow application

Build a small system that moves an operational request from capture to resolution.

Problem:

A small organisation tracks service requests through messages and spreadsheets. Staff need one place to record a request, assign ownership, update status and review overdue work.

Minimum version:

- Create, view, update and close a request.
- Store users, requests, statuses and timestamps in related tables.
- Validate required fields and reject invalid state changes.
- Filter requests by status, owner and age.
- Show loading, empty, success and failure states.
- Seed sample data and document local setup.
- Add one test around a central workflow rule.
- Record an audit entry when status changes.

Recommended stack:

C# + .NET + SQL Server or PostgreSQL + JavaScript/HTML/CSS + Git

Evidence to publish:

Evidence	Standard
Working application	Main flow, sample data and controlled failures
Repository	Staged commits, safe configuration and setup guide
Architecture note	Data model, layer boundaries and validation rules
Verification	Tests plus a documented normal and invalid run

Stretch work: role permissions, notifications, reporting view or a Docker-based local setup.

PROJECT BLUEPRINT 2

Application tracker interface

Build an interface that helps a job seeker manage opportunities, deadlines, status changes and recurring skill gaps.

Minimum version:

- Responsive list and detail views.
- Create, edit, filter and sort application records.
- Loading, empty, error and confirmation states.
- Accessible labels, keyboard navigation and visible focus.
- A small reusable component system.
- Data from a mock service, local API or public test endpoint.
- At least one interaction test around a central workflow.
- A clear mobile layout with no hidden essential action.

Recommended stack:

JavaScript + HTML/CSS -> TypeScript + React + Git

Evidence to publish:

Evidence	Standard
Behaviour	Short demo of the main workflow and a failed request
Code	Typed components, focused state logic and readable API handling
Quality	Accessibility notes and interaction tests
Decision	Explanation of component boundaries and responsive choices

Stretch work: saved filters, stage analytics, offline-friendly behaviour or a documented performance improvement.

PROJECT BLUEPRINT 3

Service request API

Build a service whose contracts, validation and failure behaviour can be inspected without a complex interface.

Problem:

Several client applications need a shared service for logging requests, assigning owners and recording status history. The service must preserve data integrity and return consistent errors.

Minimum version:

- REST endpoints for requests, users and status changes.
- Relational persistence with migrations or schema scripts.
- Input validation and consistent error responses.
- Server-enforced status-transition rules.
- Structured logging and central error handling.
- Automated tests for central rules and invalid requests.
- Seeded data and sample requests.
- One simulated external dependency with defensive handling.

Recommended stack:

C# + .NET + SQL. Java + SQL is a valid alternate; add Spring when repeated target adverts support it.

Evidence to publish:

Evidence	Standard
Contract	OpenAPI file, endpoint reference or complete request examples
Data	Schema diagram and explanation of relationships
Reliability	Tests, logs and controlled-failure examples
Operation	Repeatable startup, seeded data and environment guidance

Stretch work: authentication, idempotent operations, background processing or containerised setup.

PROJECT BLUEPRINT 4

Support ticket classifier

Build a focused intelligent feature with an evaluation method and a controlled fallback.

Problem:

A support team receives text requests that require routing by category and urgency. Build a service that proposes a category while preserving human review.

Minimum version:

- A labelled sample dataset with sensitive information removed.
- A baseline using rules, a small model or an approved API.
- An endpoint or interface that returns a proposed category.
- Evaluation on held-out examples with a suitable metric.
- A confidence threshold and review path for uncertain output.
- Logs that avoid private source text by default.
- Input validation and controlled service failures.
- An error-analysis note covering common misclassification patterns.

Recommended stack:

Python + API framework + structured data + tests + Git

Evidence to publish:

Evidence	Standard
Software	Working service, validation and failure paths
Evaluation	Dataset notes, baseline, metric and error analysis
Ownership	Category, threshold and fallback decisions
Limits	Known failure cases, privacy choices and next work

Stretch work: compare approaches, monitor drift or add a lightweight review interface.

PACKAGE AND APPLY

Project completion standard

A project is ready for applications when another person can inspect the result and understand your contribution.

Evidence layer	Required content
Working result	Main flow runs, expected failures receive handling and sample data exists.
Repository	Readable structure, traceable commits, safe configuration and clear ownership.
Technical explanation	Problem, architecture, stack, setup, decisions and current limits.
Verification	Tests, screenshots, demo, sample requests or measured output suited to the lane.

README structure:

- 1 Project purpose and intended user.
- 2 Current scope and main capabilities.
- 3 Stack and architecture summary.
- 4 Setup and configuration.
- 5 Main workflow and test instructions.
- 6 Important technical decisions.
- 7 Known limitations and next work.
- 8 Your contribution and attribution for reused material.

Reject these weak versions:

- One bulk commit after completion.
- Broken setup or missing configuration steps.
- Copied tutorial language with unclear ownership.
- Screenshots without inspectable behaviour.
- Tool lists with no connection to implementation.
- Deployment added while the main workflow remains broken.

FINAL CHECK: Send the link to another person. The project passes when they can understand the purpose and inspect the main result without a private walkthrough.

PACKAGE AND APPLY

Translate project work into CV evidence

Use concrete action, technical context, ownership and result. Include only details you can defend.

Weak claim	Evidence-led version
Built a full-stack application.	Built a C#.NET service-request application with SQL persistence, server validation and role-aware status workflows.
Worked with React.	Built reusable React components for an application tracker, including request failures and keyboard-accessible interaction states.
Created an API.	Designed a Java REST API with relational persistence, consistent error responses and tests for invalid status transitions.
Used AI in a project.	Implemented and evaluated a support-ticket classifier, added a confidence-based review path and documented common errors.

A strong project entry contains:

- The workflow or product you built.
- The stack used for the relevant layers.
- Your own contribution and scope.
- A decision, constraint or difficult failure you handled.
- A working capability, test result or measured output.
- A link that opens to the intended evidence.

Claim check:

- Every core skill appears in a task, project or credible training context.
- Team work states your contribution accurately.
- Metrics can be traced to a real result.
- The target role is visible from the project order and skills emphasis.
- Frameworks appear with the language you actually used.
- Private employer code or data stays outside the application.

PACKAGE AND APPLY

Build a target-role application system

Search across title families, then judge the responsibilities inside each advert.

Lane	Search titles
Application and full-stack	Junior Software Developer; Application Developer; Junior Full Stack Developer; Graduate Software Engineer; Software Development Intern
Front-end	Junior Front-End Developer; Junior Web Developer; Junior React Developer; UI Developer; Web Development Intern
Back-end	Junior .NET Developer; Junior Java Developer; API Developer; Integration Developer; Junior Back-End Developer
AI-adjacent	AI Software Engineer Junior/Graduate; Python Developer; AI/Data Software Intern; Junior Software Developer AI Pathway

Score each advert against four questions:

- 1 Does the work belong to my selected lane?
- 2 Do I meet the hard access requirements?
- 3 Can my evidence support the central technical duties?
- 4 Are the location and work arrangement realistic?

Apply when the lane aligns and your evidence supports the central work. Missing a preferred tool creates a learning note. Missing a required qualification or the main programming foundation creates a genuine access gap.

Application tracker fields:

Date	Company and title	Lane fit	Hard gap	Evidence used	Outcome	Follow-up

PACKAGE AND APPLY

Turn adverts and responses into a gap review

After every focused batch, count repeated signals. One advert creates a note. Repetition changes the plan.

Use these categories:

- Access gap: qualification, location, work authorisation or experience threshold.
- Foundation gap: language, programming concepts, data or debugging.
- Lane gap: framework, interface, API, database or integration capability.
- Evidence gap: relevant work exists and the application fails to show it.
- Packaging gap: CV wording, broken link, unclear ownership or weak project order.
- Search gap: titles, filters or locations produce roles outside the intended lane.

Weekly review:

Question	Decision
Which requirement repeated most often?	
Which gap blocked a suitable application?	
Which existing evidence could close the gap through repair?	
Which new learning task has the highest application value?	
Which search term produced poor-fit roles?	
What will remain unchanged next week?	

Decision rule:

Change the build plan when the same material gap appears across several suitable adverts. Keep the lane stable when the signals are isolated or belong to unrelated roles.

YOUR 30/60/90-DAY ROUTE

Day 1 to 30: establish direction and a working slice

- Complete the starting-point and lane worksheets.
- Collect suitable adverts and confirm the stack boundary.
- Reach Applied evidence in the primary language.
- Create the repository and use Git from the first feature.
- Build one vertical slice from input to visible result.
- Record one debugging case and one important decision.

Day 31 to 60: finish the core project

- Complete the minimum workflow from a clean start.
- Add validation and controlled failure handling.
- Complete the data model, API or interface depth required by the lane.
- Add focused tests around the central rules.
- Write repeatable setup and seed sample data.
- Run an external inspection with another person.

Day 61 to 90: package, apply and improve

- Publish the stable evidence and repair broken links.
- Align the CV entry with the repository and target title.
- Start a focused application batch.
- Review recurring gaps every week.
- Add one differentiator only after the core evidence passes.
- Finish the cycle with a lane and evidence review.

PROGRESSION RULE: Advance on completion evidence. Extend the calendar when the gate still fails.

PERSONAL EXECUTION PLAN

Convert the route into protected weekly work

Use four work blocks:

- Build: implement the next requirement.
- Diagnose: test failures, inspect logs and repair defects.
- Explain: update the README, decision notes and CV evidence.
- Apply: search, submit, follow up and review gaps.

Weekly hours	Build	Diagnose	Explain	Apply
5 hours	2.5	1	0.5	1
8 hours	4	1.5	1	1.5
12 hours	6	2	1.5	2.5
Your schedule				

This week's commitment

Field	Your decision
Milestone	
Deliverable	
Completion test	
Protected time blocks	
Main risk	
Scope reduction if blocked	

End-of-week review:

- What became demonstrable?
- Which failure taught me something reusable?
- Which task consumed time without improving evidence?
- What will I finish before adding another tool?
- Which application signal changed my priority?
- What remains fixed for the next week?

RESEARCH NOTES AND VERSION POLICY

Evidence basis

SkillSignalZA reviewed a purposive set of South African entry-level-accessible Software Engineering adverts and verified the structured evidence before market analysis. The retained main sample contains 22 adverts. The customer-facing percentages in this pack use explicit advert wording from that sample.

Market evidence cutoff: 11 August 2026.

The research supports concept validation and practical direction. It cannot represent every employer, region, specialisation or hiring decision in South Africa.

How evidence was treated

- Languages, frameworks and tools were counted only when explicitly named.
- Related skills were kept separate. A framework did not supply an unstated parent language.
- AI-adjacent support was counted when AI appeared explicitly in a retained advert's job title or captured requirements.
- Duplicate and out-of-scope records were removed from the main analysis.
- Missing work-arrangement data stayed missing.
- Advert frequency informed priorities without claiming that frequency proves hiring success.
- Project documentation and presentation standards remain product judgment pending candidate testing.

Version policy

This is SkillSignalZA Software Engineering Career Map Pack V1.0, published in August 2026. A future evidence cycle may revise percentages, route priorities or examples. This edition remains tied to the stated market cutoff.

Final route check

- Lane selected.
- Stack boundary written.
- Starting route selected.
- Project chosen.
- Completion gate understood.
- Weekly schedule protected.
- Application tracker ready.
- First review date recorded.

Copyright 2026 SkillSignalZA. Licensed to the purchaser for personal career planning. Redistribution or resale is prohibited.